

1. Problem Statement

Organizations maintain large volumes of inventory data — often thousands or millions of records — distributed across multiple warehouses. Employees and stakeholders frequently need quick answers to simple operational questions, such as:

- How many Dell laptops are available?
- Which warehouse has the maximum stock?
- Which items are running low?
- Should this item be reordered?

Today, answering these questions requires logging into an ERP system, navigating dashboards, applying filters, and understanding database terminology. This process is slow, requires training, and creates unnecessary friction between the user and the data they need. The objective of this project is to eliminate this friction entirely by allowing users to interact with the inventory system using natural speech — simply asking a question aloud and receiving an immediate spoken response.

2. Proposed Solution

The proposed system is an AI-powered Inventory Assistant built using **Microsoft Copilot Studio**, **FastAPI**, **PostgreSQL**, **Faster-Whisper**, and **Piper**. Unlike traditional chatbots that rely on hardcoded conversational flows and rigid menu structures, this system uses an LLM-driven agent to understand free-form spoken questions.

At a conceptual level, the backend converts speech into text, Copilot Studio interprets the user's intention, selects the appropriate inventory tool, queries the backend for data, generates a human-friendly response, and finally converts that response back into speech. The entire experience feels like a natural conversation rather than a software interaction.

3. Design Philosophy

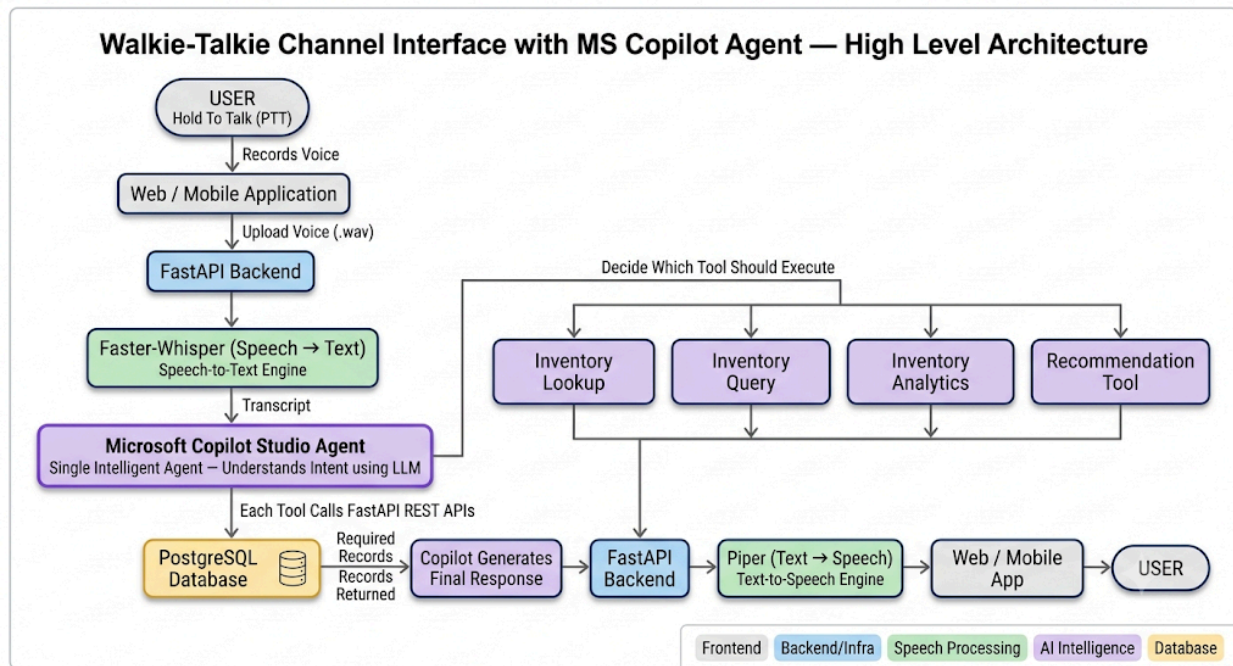
The central architectural decision behind this system is a strict separation between **intelligence** and **infrastructure**:

The intelligence lives entirely in Copilot Studio. The backend is not another AI layer.

This has an important consequence: the FastAPI backend never decides which tool to use, which SQL query to generate, whether a request is a lookup or an analytics operation, or which API should be called. All of these decisions belong entirely to Microsoft Copilot Studio. The backend's only job is to provide generic, reusable infrastructure — speech conversion and database access — while Copilot Studio handles all reasoning and decision-making.

This separation makes the system significantly simpler to build, easier to maintain, and reusable across different clients without redesigning the core logic.

4. High-Level Architecture



The system follows a linear, one-way pipeline connecting voice input to voice output:

1. The **user** presses a Hold-To-Talk (PTT) button and speaks a question.
2. The **Web/Mobile Application** records the audio and uploads it as a **.wav** file.
3. The **FastAPI Backend** receives the audio and passes it to **Faster-Whisper**, which converts speech into a text transcript.
4. The transcript is forwarded to the **Microsoft Copilot Studio Agent**.
5. Copilot Studio uses its LLM to understand the user's intent and decide which internal tool should execute the request.
6. Copilot Studio selects one of four available tools: **Inventory Lookup**, **Inventory Query**, **Inventory Analytics**, or **Recommendation Tool**.
7. The selected tool calls the corresponding **FastAPI REST API**.
8. FastAPI queries the **PostgreSQL Database** and returns the required records.
9. Copilot Studio receives the structured data and generates a natural, conversational final response.
10. The response text is sent back to **FastAPI**, converted into speech using **Piper**.
11. The generated audio and text are returned to the **Web/Mobile App** and delivered to the **user**.

This flow guarantees a clean division of labor: the backend never interprets natural language, and Copilot Studio never touches the database directly — every data access happens through a well-defined REST API.

5. System Components

5.1 Frontend

The frontend is intentionally lightweight and contains no business logic. Its responsibilities are limited to:

- Providing a Hold-To-Talk (PTT) button for recording voice input
- Offering a chat interface for displaying conversation history
- Uploading recorded audio to the backend
- Playing back the audio response
- Displaying the final text response

Once the audio is sent to the backend and the response is displayed, the frontend's job is complete.

5.2 FastAPI Backend

The backend is explicitly **not** the intelligent component of the system — it is purely an infrastructure layer. Its responsibilities include:

- Receiving recorded audio from the frontend
- Running Faster-Whisper to transcribe speech into text
- Communicating with Microsoft Copilot Studio
- Exposing REST APIs for inventory operations
- Connecting to and querying PostgreSQL
- Converting Copilot's final text response into speech using Piper

Critically, the backend never decides whether a request is a Lookup, an Analytics query, or a Recommendation — it simply exposes generic APIs and executes whichever specific request it receives.

5.3 Faster-Whisper (Speech-to-Text)

The backend sends recorded audio to Faster-Whisper for transcription. For example, an audio clip is converted as follows:

- **Input:** Audio recording

- **Output (Transcript):** "How many Dell Latitude laptops are available?"

Only after this transcription is complete does the backend forward the text to Copilot Studio for interpretation.

5.4 Microsoft Copilot Studio (Decision-Making Layer)

This is the most important component in the entire system. Copilot Studio is responsible for:

- Understanding natural language input
- Detecting the user's intent
- Deciding which inventory tool should execute
- Generating structured, SQL-oriented parameters through tool calls
- Invoking the correct backend API
- Generating a natural, conversational final response

Importantly, the backend never tells Copilot Studio which tool to use — Copilot makes this decision entirely on its own, based on the LLM's understanding of the transcript.

5.5 PostgreSQL Database

PostgreSQL serves as the single source of truth for all operational data, storing:

- Products
- Warehouses
- Suppliers
- Categories
- Stock levels
- Purchase history
- Inventory movements

Each client maintains their own independent PostgreSQL database. For example, Client A may deal in Dell, HP, and Lenovo products; Client B may manage medical equipment; and Client C may handle automobile parts. The Copilot Agent itself never changes across clients — only the backend's connected database changes, making the system highly reusable.

6. Inventory Tools

A single Copilot Agent contains four distinct tools. These tools are not business logic modules in themselves — they act as AI-powered connectors between the LLM and the backend's REST APIs.

6.1 Tool 1 — Inventory Lookup

Purpose: Retrieve inventory information for specific products.

Example queries:

- How many Dell laptops are available?
- Do we have HP EliteBooks?
- Show inventory in the Bangalore warehouse.
- Find monitors under ₹20,000.

Copilot extracts relevant parameters — product, warehouse, price, and category — from the user's speech, then calls `POST /api/inventory/lookup`. The backend simply queries PostgreSQL and returns the matching records.

6.2 Tool 2 — Inventory Query Handling

Purpose: Handle more flexible inventory questions involving filtering, grouping, and relationships.

Example queries:

- Which items are running low?
- Which products are out of stock?
- Show inventory added this month.
- Which supplier provided this item?
- Which warehouse has the highest stock?

Here, the LLM interprets filtering conditions, grouping requirements, and relational context, then calls `POST /api/inventory/query` with the appropriate structured parameters. The backend does not interpret any English text — it simply executes the pre-structured query safely against PostgreSQL.

6.3 Tool 3 — Inventory Analytics

Purpose: Generate analytical insights from inventory data.

Example queries:

- Highest stock
- Average stock
- Top 10 products
- Category distribution
- Warehouse summary

Copilot determines that an analytical operation is required and calls `POST /api/inventory/analytics`. The backend performs the SQL aggregation and returns numerical results, which Copilot then converts into a natural-language explanation.

6.4 Tool 4 — Inventory Recommendation

Purpose: Provide intelligent business suggestions based on inventory data.

Example queries:

- Should we reorder?
- Are there alternative products?
- Which items are likely to run out?
- Suggest restocking priorities.

Copilot decides that recommendation logic is required and calls `POST /api/inventory/recommendation`. The backend executes relevant business rules or SQL queries and returns structured recommendation data, which Copilot then explains conversationally to the user.

7. Why PostgreSQL Instead of Storing Data in Copilot

Copilot Studio is not designed to function as the operational inventory database, and storing all data within it would be impractical. Large enterprises may have:

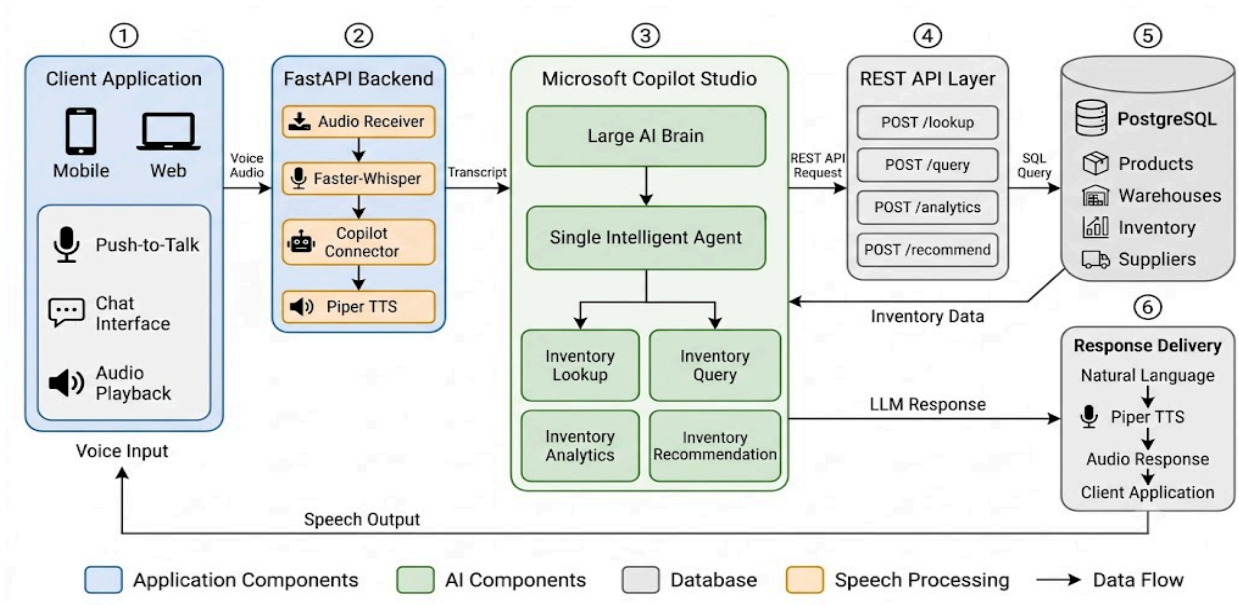
- 50 million products
- 100 warehouses
- 500 GB of database size

Sending this volume of data directly to an LLM is simply not feasible. Instead, the system follows a lightweight, scalable pattern:

Copilot → Backend API → SQL Query → 10 Rows Returned → Copilot Explains

The LLM only ever sees the small number of records relevant to a specific query, regardless of how large the underlying dataset is. This keeps response times fast and keeps the system scalable to enterprise-level data volumes.

8. End-to-End Example Flow



To illustrate the complete process, consider a user asking: *"How many Dell laptops are available?"*

Step	Action
1	User presses Hold-To-Talk and records their voice.
2	Frontend uploads audio.wav to FastAPI.
3	FastAPI sends the audio to Faster-Whisper, producing the transcript: "How many Dell laptops are available?"
4	FastAPI sends the transcript to Microsoft Copilot Studio.
5	Copilot identifies Intent = Inventory Lookup, Entity = "Dell Laptop," and extracts parameters such as warehouse, category, and price filters.
6	Copilot decides that the Inventory Lookup Tool should execute — this decision is made entirely by the LLM.
7	The tool calls POST /api/inventory/lookup with a structured request: {"product": "Dell Laptop", "warehouse": "Bangalore"}.
8	The backend queries PostgreSQL and returns: {"product": "Dell Latitude 5440", "stock": 45, "warehouse": "Bangalore", "status": "Available"}.
9	Copilot receives this structured data and generates a natural sentence: "Dell Latitude 5440 has 45 units available in the Bangalore warehouse."
10	FastAPI receives this response text and calls Piper to generate speech audio.
11	The frontend receives both text and audio, displays the message, and plays the spoken response — completing the conversation.

9. Technology Stack

Layer	Technology	Responsibility
Frontend	Web / Mobile Application	Hold-To-Talk interface, chat display, audio playback
Speech-to-Text	Faster-Whisper	Converts recorded voice into text transcript
Backend Framework	FastAPI	Hosts REST APIs, orchestrates infrastructure calls
Intelligence / Orchestration	Microsoft Copilot Studio	Understands intent, selects tools, generates responses
Database	PostgreSQL	Stores products, warehouses, suppliers, stock, and history
Text-to-Speech	Piper	Converts Copilot's final text response into speech
Communication	REST APIs	Connects Copilot tools to backend endpoints

10. Why This Architecture Is Production-Ready

This architecture enforces a clear separation of responsibilities across every layer of the system:

- The **frontend** handles only user interaction.
- **FastAPI** handles infrastructure concerns: speech processing, API hosting, database connectivity, and text-to-speech conversion.
- **PostgreSQL** stores all client-specific inventory data.
- **Copilot Studio** provides the intelligence: understanding language, selecting tools, orchestrating API calls, and generating conversational responses.

Because intelligence is isolated entirely in Copilot Studio and data resides entirely in PostgreSQL, the same Copilot Agent can be reused across different clients with minimal effort. Onboarding a new customer typically requires only pointing the backend at a different PostgreSQL database or adapting the repository layer — not redesigning the Copilot agent itself. This separation of concerns makes the system easier to maintain, scale, and deploy in real enterprise environments.